

Development and Architectural Report

JournaBuddy: A Multi-Agent AI Research Intelligence Platform

Abdullah Al Mamun

B.Sc. Software Engineering, Daffodil International University

M.Sc. Software Engineering (in progress), TU Wien

mamun.swe.de@gmail.com

ORCID: [0009-0006-7473-0024](https://orcid.org/0009-0006-7473-0024)

July 18, 2026

Abstract

The peer-review and pre-submission evaluation phase of academic publishing is historically bottlenecked by manual review processes, citation verification, and rigorous plagiarism checks. This report details the full development lifecycle, architectural design, and algorithmic implementation of **JournaBuddy**, an advanced AI-powered research intelligence platform.

Contents

1	System Architecture and Flow	2
1.1	Use Case Diagram	2
1.2	System Flowchart	2
2	Core Algorithms	2
2.1	Robust LLM JSON Extraction Algorithm	2
3	Parallel Agent Implementation	3
4	Conclusion	3

1 System Architecture and Flow

1.1 Use Case Diagram

Figure 1 illustrates the core interactions between the Researcher and the JournaBuddy ecosystem.

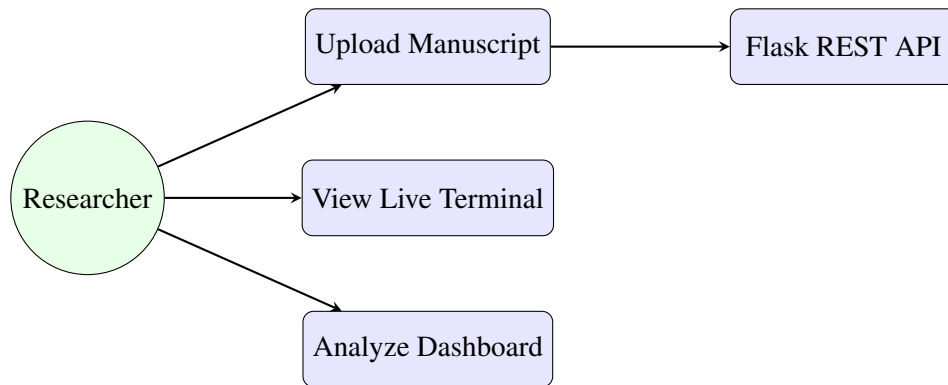


Figure 1: System Use Case Diagram

1.2 System Flowchart

Figure 2 outlines the asynchronous data flow from upload to final BI rendering.

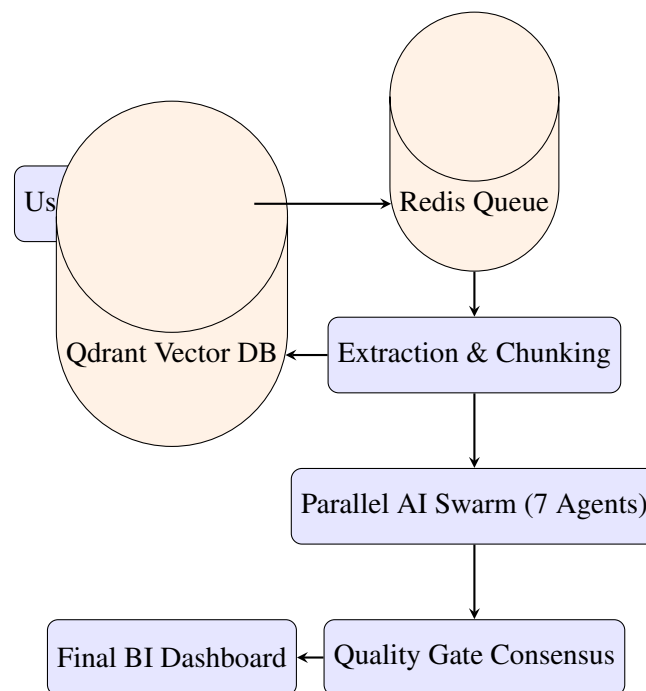


Figure 2: Asynchronous Pipeline Flowchart

2 Core Algorithms

2.1 Robust LLM JSON Extraction Algorithm

To handle unstructured conversational output from Large Language Models (LLMs), a two-step fallback algorithm (Algorithm 1) was engineered.

```

1 def extract_json(raw_response):
2     # Step 1: Regex Block Extraction
3     if contains_markdown_blocks(raw_response):
4         extracted = regex_extract_backticks(raw_response)
5         return json.loads(extracted)
6
7     # Step 2: Bracket Parsing Fallback
8     else:
9         start_idx = raw_response.find('{')
10        end_idx = raw_response.rfind('}')
11        sliced = raw_response[start_idx : end_idx + 1]
12        return json.loads(sliced)

```

Listing 1: Robust Two-Step JSON Extraction Fallback

3 Parallel Agent Implementation

The system utilizes a Thread Pool Executor to run independent AI agents simultaneously, as visualized in Figure 3. This significantly reduces the inference latency.

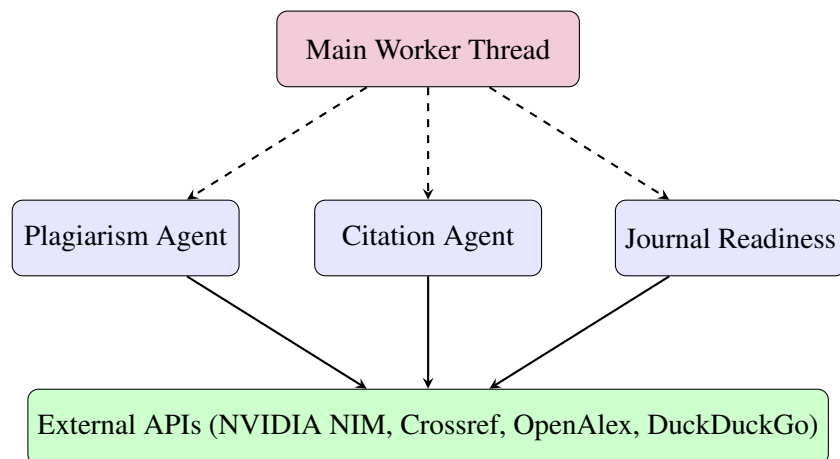


Figure 3: Concurrent Multi-Agent Swarm Architecture

4 Conclusion

JournaBuddy integrates complex NLP algorithms, real-time API integrations, and robust concurrency mechanisms to deliver a state-of-the-art pre-submission screening tool.